

IoT Ağ Trafiğinde Saldırı Tipi Sınıflandırması

Büyük Veri Projesi Raporu

Veri Seti Edge-IIoTset Cyber Security Dataset
Teknolojiler Kafka, Spark, Delta Lake, MLflow, Streamlit
Tarih Mayıs 2025

Proje Ekibi

Öğrenci No	Ad Soyad
220201018	Abdulrahman Dülek
220201061	Eyüp Ensar Kara
220201083	Yunus Hanifi Öztürk

İçindekiler

1 Giriş	3
1.1 Proje Kapsamı	3
2 Veri Seti	3
2.1 Genel Bilgiler	3
2.2 Sınıf Dağılımı	3
2.3 Veri Seti Özellikleri	4
3 Sistem Mimarisi	4
3.1 Bileşenler	5
3.2 Docker Container'lar	5
3.3 Veri Akış Mimarisi	5
4 Veri Pipeline'ı	5
4.1 Kafka Producer	5
4.2 Medallion Architecture (Delta Lake)	6
4.2.1 Bronze Katmanı (Ham Veri)	6
4.2.2 Silver Katmanı (Temiz Veri)	6
4.2.3 Gold Katmanı (ML-Ready)	6
5 Feature Engineering	7
5.1 Feature Açıklamaları	7
6 Makine Öğrenmesi Modelleri	7
6.1 Ortak Pipeline	7
6.2 Eğitilen Modeller	8
6.2.1 Model 1: Logistic Regression	8
6.2.2 Model 2: Decision Tree	8
6.2.3 Model 3: Random Forest	9
6.2.4 Model 4: GBT (OneVsRest)	9
6.2.5 Model 5: Naive Bayes	9
6.3 Değerlendirme Metrikleri	9
7 MLflow Deney Takibi	10
8 Streamlit Dashboard	10
9 Sonuç	11
9.1 Özet	11
9.2 Karşılaşılan Zorluklar	11

Kaynakça

12

1 Giriş

Bu proje, **Edge-IIoTset** veri setini kullanarak IoT ağ trafiği üzerinde çok sınıflı (multi-class) saldırı tipi sınıflandırması yapan uçtan uca bir büyük veri pipeline'ı sunar. Kafka ile gerçek zamanlı akış, Spark ile dağıtık işleme, Delta Lake ile katmanlı depolama, 5 ML modeli, MLflow ile deney takibi ve Streamlit dashboard bileşenleri Docker Compose ile orkestre edilir.

Amaç: Gelen her ağ paketinin saldırı türünü (Normal, DDoS, Port Scanning, MITM, Ransomware vb.) yüksek doğrulukla sınıflandırmak.

1.1 Proje Kapsamı

- Gerçek zamanlı veri toplama ve akış (Kafka → Spark Structured Streaming)
- Medallion Architecture ile katmanlı veri depolama (Bronze → Silver → Gold)
- 5 farklı makine öğrenmesi modelinin eğitimi ve karşılaştırılması
- MLflow ile deney takibi ve model versiyonlama
- Streamlit ile interaktif izleme ve analiz dashboard'u
- Tüm bileşenlerin Docker Compose ile orkestrasyonu

2 Veri Seti

2.1 Genel Bilgiler

Kaynak	Kaggle (Mohamed Amine Ferrag)
Dosya	ML-EdgeIIoT-dataset.csv (~82 MB)
Satır	~200 000 ağ paketi kaydı
Kolon	63 (TCP, UDP, HTTP, DNS, MQTT, Modbus vb.)
Sınıf	14+ saldırı tipi + Normal
Hedef	Attack_type (çok sınıflı) / Attack_label (ikili)

Her satır bir ağ paketini temsil eder. Paket yalnızca bir protokol kullandığından, diğer protokol kolonları doğal olarak 0'dır (eksik değer değil, aktivite yok).

2.2 Sınıf Dağılımı

Veri setindeki başlıca saldırı türleri şunlardır:

Saldırı Tipi	Açıklama
Normal	Meşru ağ trafiği
DDoS_UDP	UDP tabanlı dağıtık servis engelleme
DDoS_TCP	TCP tabanlı dağıtık servis engelleme
DDoS_ICMP	ICMP tabanlı dağıtık servis engelleme
DDoS_HTTP	HTTP tabanlı dağıtık servis engelleme
Port Scanning	Açık port tarama saldırısı
Vulnerability Scanner	Zafiyet tarama saldırısı
Password Brute Force	Parola kaba kuvvet saldırısı
OS Fingerprinting	İşletim sistemi parmak izi tespiti
Ransomware	Fidye yazılımı ağ trafiği
MITM (ARP Spoofing)	Ortadaki adam saldırısı

Veri setinde ciddi bir **sınıf dengesizliği** mevcuttur. Bazı saldırı türleri on binlerce kayda sahipken, bazıları birkaç yüz ile sınırlıdır. Bu dengesizlik, model eğitiminde *balanced class weighting* ile ele alınmaktadır (bkz. Bölüm 6).

2.3 Veri Seti Özellikleri

Kolonlar ağ protokollerine göre gruplanabilir:

- **TCP:** `tcp.flags`, `tcp.len`, `tcp.seq`, `tcp.ack`, `tcp.checksum`, bağlantı durumları (SYN/FIN/RST)
- **UDP:** `udp.port`, `udp.length`, `udp.stream`
- **HTTP:** `http.request`, `http.response`, `http.content_length`
- **DNS:** `dns.qry.type`, `dns.qry.name.len`
- **MQTT/Modbus:** IoT'ye özgü protokol alanları

3 Sistem Mimarisi

Tüm bileşenler `docker compose up -d` ile tek komutla ayağa kalkar. Sistem 9 Docker container'dan oluşur ve `bigdata-net` adlı bir bridge network üzerinden haberleşir.

3.1 Bileşenler

Bileşen	Teknoloji	Port
Mesaj Kuyruğu	Kafka 3.4.1 + Zookeeper 3.8.3	9092, 29092
Dağıtık İşleme	Apache Spark 3.4.2 (master)	8080, 7077
Spark Worker	Apache Spark 3.4.2 (worker)	8081
Depolama	Delta Lake 2.4.0	—
ML Takip	MLflow (SQLite backend)	5000
Dashboard	Streamlit	8501
Notebook	JupyterLab	8888
API Sunucu	Flask (Dashboard API)	5001

Spark cluster: 1 master + 1 worker (2 GB RAM, 2 core).

3.2 Docker Container'lar

Container	Görev
zookeeper	Kafka broker koordinasyonu
kafka	Mesaj kuyruğu servisi
kafka-producer	CSV'yi JSON'a çevirip Kafka'ya gönderir
spark-master	Spark cluster koordinatörü
spark-worker-1	Spark iş yükü çalıştırıcı
mlflow-server	Deney takip sunucusu
jupyter-lab	İnteraktif notebook ortamı
dashboard-api	Delta Lake REST API
streamlit-dashboard	Web tabanlı izleme arayüzü

3.3 Veri Akış Mimarisi

Uçtan uca veri akışı şu sırayla ilerler:

1. **Producer:** CSV → JSON → Kafka topic
2. **Spark Streaming:** Kafka → Bronze (ham) → Silver (temiz) → Gold (ML-ready)
3. **ML Eğitimi:** Gold Delta → 5 model eğitimi → MLflow'a loglama
4. **Dashboard:** Delta Lake + MLflow verileri → Streamlit görselleştirme

4 Veri Pipeline'ı

4.1 Kafka Producer

CSV dosyasını satır satır okuyarak her kaydı JSON formatına çevirir ve `iot-network-traffic` topic'ine gönderir. Her mesaja aşağıdaki metadata alanları eklenir:

- **timestamp** — UTC zaman damgası (ISO 8601)
- **flow_id** — Kaynak IP:port → Hedef IP:port + zaman damgasından oluşan benzersiz akış kimliği
- **source_ip, dest_ip** — Kaynak ve hedef IP adresleri
- **attack_type** — Saldırı tipi etiketi (Gold katmanına kadar taşınır)

Mesajlar `source_ip` üzerinden key'lenir; böylece aynı kaynaktan gelen paketler aynı partition'a yönlendirilir.

4.2 Medallion Architecture (Delta Lake)

Veriler üç katmanlı Medallion Architecture ile işlenir. Her katman Spark Structured Streaming ile 30 saniyelik micro-batch'ler halinde çalışır.

4.2.1 Bronze Katmanı (Ham Veri)

Kafka'dan okunan ham JSON payload'u hiçbir dönüşüm yapılmadan Delta Lake'e yazılır. Yazma sırasında `ingestion_time` alanından türetilen `yıl/ay/gün` partition'ları oluşturulur. Bu katman veri kaybını önlemek için bir "güvenli kopya" işlevi görür.

4.2.2 Silver Katmanı (Temiz Veri)

Bronze'dan okunan veriler aşağıdaki adımlarla temizlenir:

1. **JSON Parse:** Ham JSON string'i, CSV şemasına uygun olarak parse edilir.
2. **Kolon Yeniden Adlandırma:** Nokta içeren kolon isimleri (`tcp.ack` → `tcp_ack`) alt çizgiyle değiştirilir; Spark'ın nested struct yorumlamasını önler.
3. **Gereksiz Kolon Silme:** 8 kolon kaldırılır (`arp_dst_proto_ipv4`, `arp_opcode`, `arp_hw_size`, `arp_src_proto_ipv4`, `icmp_checksum`, `icmp_seq_le`, `icmp_transmit_timestamp`, `icmp_unused`).
4. **Inf Temizleme:** Tüm sayısal kolonlardaki $\pm\infty$ değerleri `null`'a çevrilir.
5. **Eksik Değer Doldurma:** Sayısal kolonlar 0 ile, string kolonlar "unknown" ile doldurulur.
6. **Tip Dönüşümü:** `Attack_label` → integer, port kolonları → integer.
7. **Duplikat Kaldırma:** Aynı `flow_id`'ye sahip kayıtlar `dropDuplicates` ile elenir.

4.2.3 Gold Katmanı (ML-Ready)

Silver verisine 5 yeni feature eklenerek (bkz. Bölüm 5) makine öğrenmesine hazır son tablo üretilir. Bu katmandaki veri, ML script'leri tarafından batch modda okunur.

5 Feature Engineering

Silver verisine eklenen 5 özellik, IoT ağ trafiğindeki saldırı imzalarını yakalamak üzere alan bilgisiyle (domain knowledge) tasarlanmıştır. Tüm feature'lar `withColumn` ile üretildiğinden Structured Streaming ile tam uyumludur.

#	Feature	Formül	Hedef Saldırı
1	<code>traffic_asymmetry_ratio</code>	$\frac{tcp_ack}{tcp_seq+1}$	DDoS Flood
2	<code>pkt_size_cv</code>	$\frac{tcp_len}{ seq-ack +1}$	Port Scan
3	<code>flow_intensity</code>	$tcp_len \times tcp_flags$	DDoS UDP
4	<code>iat_regularity</code>	$\frac{tcp_checksum}{tcp_len+1}$	Brute Force
5	<code>conn_efficiency</code>	$\frac{syn}{fin+rst+1}$	SYN Flood

5.1 Feature Açıklamaları

1. **traffic_asymmetry_ratio:** Normal TCP trafiği simetrik bir ACK/SEQ akışına sahiptir. DDoS saldırılarında saldırgan çok sayıda SYN gönderir ancak el sıkışmayı tamamlamaz; bu durum ACK/SEQ oranını bozar.
2. **pkt_size_cv:** Paket boyutu ile bağlantı durumu (`seq-ack` farkı) arasındaki oran, port tarama saldırılarında bozulur. Tarayıcılar küçük paketlerle çok sayıda bağlantı açar.
3. **flow_intensity:** Paket boyutu ve bayrak sayısını tek bir metrikte birleştirir. Volumetrik saldırılarda her iki değer de anormal seviyelere ulaşır.
4. **iat_regularity:** TCP checksum ile paket boyutu oranı, crafted (yapay üretilmiş) paketleri tespit eder. Saldırı araçları küçük payload ile yüksek header overhead üretir.
5. **conn_efficiency:** Normal bir bağlantıda `SYN→veri→FIN/RST` akışı vardır ($SYN/FIN \approx 1$). Port tarama ve SYN flood saldırıları bağlantıyı tamamlamadığından bu oran çok yükselir.

Tüm feature'larda $\pm\infty$ ve NaN değerleri 0.0 ile değiştirilir.

6 Makine Öğrenmesi Modelleri

6.1 Ortak Pipeline

Tüm modeller aynı ön işleme pipeline'ını paylaşır:

1. **Veri Okuma:** Gold Delta tablosundan batch okuma
2. **Label Encoding:** `StringIndexer` ile `Attack_type` → sayısal label (en sık sınıf=0)

3. **Feature Vektörleme:** `VectorAssembler` ile tüm sayısal kolonlar tek bir `features` vektörüne birleştirilir. NaN/null değerler 0.0 ile doldurulur.
4. **Sınıf Ağırlıkları:** Dengesizliği telafi etmek için `balanced class weight`:

$$w_c = \frac{N}{K \cdot N_c}$$

burada N = toplam satır, K = sınıf sayısı, N_c = ilgili sınıfın satır sayısı. Her satıra `classWeight` kolonu eklenir.

5. **Veri Bölme:** Label bazlı stratified `sampleBy` ile %80/%20 train/test split (seed=42). Sınıf dağılımı her iki sette korunur.

6.2 Eğitilen Modeller

#	Model	Özellik
1	Logistic Regression	Multinomial, <code>StandardScaler</code> , <code>elasticNet CV</code>
2	Decision Tree	<code>maxDepth</code> / <code>minInstances</code> / <code>impurity CV</code> , Gini & Entropy
3	Random Forest	50–200 ağaç, bagging ensemble
4	GBT (OneVsRest)	N binary GBT, boosting ensemble
5	Naive Bayes	<code>MinMaxScaler</code> , multinomial/complement

Tüm modeller `CrossValidator` ile **F1-Score** optimizasyonu yapar.

6.2.1 Model 1: Logistic Regression

Multinomial logistic regression, `StandardScaler` ile feature ölçekleme sonrası eğitilir. Cross validation grid'i:

- `regParam`: [0.001, 0.01, 0.1] — L2 regularization gücü
- `elasticNetParam`: [0.0, 0.5, 1.0] — L1/L2 karışım oranı (0=Ridge, 1=Lasso)
- Fold: 3 (full), 2 (quick)

Eğitim sonrası `coefficientMatrix`'ten sınıflar arası ortalama |katsayı| ile global feature importance hesaplanır.

6.2.2 Model 2: Decision Tree

Yorumlanabilir bir model olarak karar ağacı eğitilir. CV grid'i:

- `maxDepth`: [5, 10, 15, 20]
- `minInstancesPerNode`: [1, 5, 10]
- `impurity`: [gini, entropy]

Eğitim sonrası ağaç derinliği, düğüm/yaprak sayısı ve `toDebugString` ile ağaç yapısı analiz edilir.

6.2.3 Model 3: Random Forest

Bagging tabanlı ensemble yöntemi. CV grid'i:

- **numTrees:** [50, 100, 200] — orman büyüklüğü
- **maxDepth:** [10, 15, 20]
- **minInstancesPerNode:** [1, 5]
- **Fold:** 5 (full), 3 (quick)

Her ağacın `featureImportances` vektörü birleştirilerek global önem skoru üretilir.

6.2.4 Model 4: GBT (OneVsRest)

Spark MLlib'in `GBTClassifier`'ı doğrudan multi-class desteklemediğinden, `OneVsRest` wrapper kullanılır: K sınıf $\rightarrow K$ adet binary GBT. CV grid'i:

- **maxIter:** [20, 50]
- **maxDepth:** [3, 5, 7]
- **stepSize:** 0.1 (sabit)

Toplam fit sayısı = $\text{grid} \times \text{fold} \times K$ olduğundan eğitim süresi diğer modellere göre önemli ölçüde uzundur.

6.2.5 Model 5: Naive Bayes

Naive Bayes negatif feature değerlerini kabul etmediğinden, `MinMaxScaler` ile $[0, 1]$ aralığına ölçeklenir. CV grid'i:

- **smoothing:** [0.5, 1.0, 2.0] — Laplace düzleştirme parametresi
- **modelType:** [multinomial, complement]

Not: Spark'ın NaiveBayes implementasyonu `weightCol` desteklemez; sınıf dengesizliği modelin doğal smoothing mekanizması ile kısmen telafi edilir. Feature importance olarak `theta` matrisinden (log-likelihood) sınıflar arası varyans hesaplanır.

6.3 Değerlendirme Metrikleri

Her model aşağıdaki metriklerle değerlendirilir:

- **Accuracy:** Toplam doğru tahmin oranı
- **Weighted F1-Score:** Sınıf dengesizliğini dikkate alan harmonik ortalama
- **Weighted Precision:** Ağırlıklı kesinlik

- **Weighted Recall:** Ağırlıklı duyarlılık
- **Log Loss:** Olasılık tahminlerinin kalitesi (varsa)
- **Per-class P/R/F1:** Her sınıf için ayrı precision, recall, F1
- **$N \times N$ Confusion Matrix:** Tüm sınıflar arası karışıklık tablosu

7 MLflow Deney Takibi

Tüm modeller `iot_intrusion_detection` adlı tek bir MLflow experiment altında toplanır. Her eğitim sonrası aşağıdaki bilgiler otomatik olarak loglanır:

- **Metrikler:** accuracy, f1_score, precision, recall, log_loss, per-class precision/recall/f1, per-class accuracy, confusion matrix satır toplamaları
- **Parametreler:** tüm hiperparametreler (regParam, maxDepth, numTrees vb.), CV fold sayısı, grid büyüklüğü, toplam fit sayısı, `weightCol` bilgisi
- **Artifact'lar:**
 - `confusion_matrix.csv` — $N \times N$ karışıklık matrisi
 - `feature_importance.csv` — Feature önem sıralaması
 - Feature importance bar chart (PNG)
 - `label_index_mapping.csv` — Label index $\rightarrow$ sınıf ismi eşlemesi
 - `cv_results.csv` — Cross validation grid sonuçları
 - Spark PipelineModel (`spark-model/` dizini)
- **Tag'ler:** `classification_type=multiclass`, `num_classes`, `model_type`, `model_index`, `label_column=Attack_type`

Altyapı: Backend store olarak SQLite (`mlflow.db`), artifact store olarak dosya sistemi kullanılır. MLflow UI <http://localhost:5000> adresinden erişilebilir. Bu yapı sayesinde modeller arasında metrik karşılaştırması, parametre analizi ve model seçimi kolayca yapılabilir.

8 Streamlit Dashboard

6 sayfalık interaktif web arayüzü, Delta Lake ve MLflow verilerini gerçek zamanlı olarak görselleştirir:

#	Sayfa ve İçerik
1	Genel Bakış — Sistem durumu, container bilgileri, pipeline özeti, toplam kayıt sayısı
2	Veri Akışı — Kafka → Bronze → Silver → Gold akışının canlı izlenmesi, katman bazlı satır sayıları, veri akış hızı
3	EDA — Sınıf dağılımı, korelasyon matrisi, sayısal kolon dağılımları, protokol bazlı analiz
4	Feature Engineering — 5 feature'ın formülleri, iş mantığı açıklamaları, istatistiksel özet
5	Model Karşılaştırma — 5 modelin F1, Accuracy, Precision, Recall bazlı karşılaştırmalı tablosu ve grafikleri
6	En İyi Model — En yüksek F1-Score'a sahip modelin detaylı analizi, confusion matrix görselleştirmesi, production tavsiyesi

Dashboard, Delta Lake verilerine `dashboard-api` (Flask) üzerinden REST çağrılarıyla; MLflow metriklerine ise doğrudan SQLite veritabanından erişir.

9 Sonuç

9.1 Özet

Bu proje, IoT ağ güvenliği için gerçek zamanlı veri toplama, temizleme, feature üretimi ve çok sınıflı saldırı sınıflandırmasını uçtan uca bir büyük veri platformu altında birleştirmiştir. Kafka ile gerçek zamanlı akış, Spark ile dağıtık işleme, Delta Lake ile katmanlı depolama, 5 farklı ML modeli, MLflow ile sistematik deney takibi ve Streamlit ile interaktif görselleştirme başarıyla entegre edilmiştir.

9.2 Karşılaşılan Zorluklar

- **Sınıf Dengesizliği:** Bazı saldırı türleri çok az örneğe sahip olduğundan, dengeli sınıf ağırlıkları ($w_c = N/K \cdot N_c$) kullanılarak bu sorun telafi edilmiştir.
- **GBT Multi-class:** Spark MLlib'in GBTCClassifier'ı doğrudan çok sınıflı sınıflandırma desteklemediğinden, OneVsRest wrapper ile K adet binary model eğitilmiştir. Bu durum eğitim süresini K kat artırmıştır.
- **Streaming + ML Uyumu:** Spark Structured Streaming ile `VectorAssembler` gibi ML nesneleri doğrudan kullanılamadığından, Gold katmanında yalnızca `withColumn` tabanlı feature üretimi yapılmış; vektörleştirme batch ML pipeline'ına bırakılmıştır.
- **Kolon İsimlendirme:** CSV'deki noktalı kolon isimleri (`tcp.ack`) Spark'ta nested struct olarak yorumlandığından, Silver katmanında tüm noktalar alt çizgiye dönüştürülmüştür.

Kaynakça

1. Ferrag, M.A. et al. “Edge-IIoTset: A New Comprehensive Realistic Cyber Security Dataset of IoT and IIoT Applications.” *IEEE Access*, vol. 10, pp. 321–332, 2022.
2. Zaharia, M. et al. “Apache Spark: A Unified Engine for Big Data Processing.” *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.
3. Armbrust, M. et al. “Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores.” *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 3411–3424, 2020.
4. Apache Kafka Documentation. <https://kafka.apache.org/documentation/>
5. MLflow Documentation. <https://mlflow.org/docs/latest/index.html>
6. Meng, X. et al. “MLlib: Machine Learning in Apache Spark.” *Journal of Machine Learning Research*, vol. 17, no. 34, pp. 1–7, 2016.
7. Streamlit Documentation. <https://docs.streamlit.io/>